

LAB EXPERIMENTS:

DYNAMIC PROGRAMMING:

1. List Operations
2. Selection Sort
3. Insertion Sort
4. Optimized Bubble Sort
5. Closest Pair of Points
6. Convex Hull
7. Travelling Salesman Problem
8. Assignment Problem
9. Knapsack
10. Bubble Sort
11. Find Maximum Element
12. Sort and Find Maximum
13. Binary Search
14. Efficient Sorting
15. String Comparison Using Stack

main.py

VisualizeRun

```
1 # List Operations
2
3 a = [10, 20, 30, 40, 50]
4
5 print("Original list:", a)
6
7 # Insert
8 a.append(60)
9 print("After append:", a)
10
11 # Insert at position
12 a.insert(2, 25)
13 print("After insert:", a)
14
15 # Delete
16 a.remove(30)
17 print("After delete:", a)
18
19 # Sort
20 a.sort()
21 print("Sorted list:", a)
22
23 # Reverse
24 a.reverse()
25 print("Reversed list:", a)
26
```

Output

```
Original list: [10, 20, 30, 40, 50]
After append: [10, 20, 30, 40, 50, 60]
After insert: [10, 20, 25, 30, 40, 50, 60]
After delete: [10, 20, 25, 40, 50, 60]
Sorted list: [10, 20, 25, 40, 50, 60]
Reversed list: [60, 50, 40, 25, 20, 10]
40 found
Length: 6

=== Code Execution Successful ===
```

main.py

Visualize

Run

```
1 def selection_sort(a):
2     n = len(a)
3
4     for i in range(n):
5         min_index = i
6
7         for j in range(i + 1, n):
8             if a[j] < a[min_index]:
9                 min_index = j
10
11        a[i], a[min_index] = a[min_index], a[i]
12
13    return a
14
15
16 a = [64, 25, 12, 22, 11]
17 print("Sorted list:", selection_sort(a))
```

Output

Sorted list: [11, 12, 22, 25, 64]

=== Code Execution Successful ===

main.py

Visualize

Run

```
1 def insertion_sort(a):
2     for i in range(1, len(a)):
3         key = a[i]
4         j = i - 1
5
6         while j >= 0 and a[j] > key:
7             a[j + 1] = a[j]
8             j -= 1
9
10        a[j + 1] = key
11
12    return a
13
14
15 a = [12, 11, 13, 5, 6]
16 print("Sorted list:", insertion_sort(a))
```

Output

Sorted list: [5, 6, 11, 12, 13]

=== Code Execution Successful ===

main.py

Visualize

Run

```
1 def optimized_bubble_sort(a):
2     n = len(a)
3
4     for i in range(n):
5         swapped = False
6
7         for j in range(0, n - i - 1):
8             if a[j] > a[j + 1]:
9                 a[j], a[j + 1] = a[j + 1], a[j]
10                swapped = True
11
12        if not swapped:
13            break
14
15    return a
16
17
18 a = [1, 2, 3, 5, 4]
19 print("Sorted list:", optimized_bubble_sort(a))
```

Output

Sorted list: [1, 2, 3, 4, 5]

=== Code Execution Successful ===

main.py

Visualize

Run

```
1 import math
2
3 def distance(p1, p2):
4     return math.sqrt((p1[0] - p2[0]) ** 2 +
5                     (p1[1] - p2[1]) ** 2)
6
7
8 def closest_pair(points):
9     min_dist = float('inf')
10    pair = None
11
12    for i in range(len(points)):
13        for j in range(i + 1, len(points)):
14            d = distance(points[i], points[j])
15
16            if d < min_dist:
17                min_dist = d
18                pair = (points[i], points[j])
19
20    return pair, min_dist
21
22
23 points = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10)]
24
25 pair, d = closest_pair(points)
26
27 print("Closest pair:", pair)
28 print("Distance:", d)
```

Output

Closest pair: ((2, 3), (5, 1))
Distance: 3.605551275463989

=== Code Execution Successful ===

main.py

Visualize

Run

```
1 def cross(o, a, b):
2     return ((a[0] - o[0]) * (b[1] - o[1]) -
3           (a[1] - o[1]) * (b[0] - o[0]))
4
5
6 def convex_hull(points):
7     points = sorted(set(points))
8
9     if len(points) <= 1:
10        return points
11
12    lower = []
13
14    for p in points:
15        while len(lower) >= 2 and cross(lower[-2], lower[-1], p) <= 0:
16            lower.pop()
17        lower.append(p)
18
19    upper = []
20
21    for p in reversed(points):
22        while len(upper) >= 2 and cross(upper[-2], upper[-1], p) <= 0:
23            upper.pop()
24        upper.append(p)
25
26    return lower[:-1] + upper[:-1]
27
```

Output

Convex Hull:
[(0, 0), (2, 0), (2, 2), (0, 2)]

=== Code Execution Successful ===

main.py

Visualize

Run

```
1 from itertools import permutations
2
3 def tsp(graph):
4     n = len(graph)
5     cities = range(1, n)
6
7     min_cost = float('inf')
8     best_path = None
9
10    for path in permutations(cities):
11        current_path = (0,) + path + (0,)
12
13        cost = 0
14
15        for i in range(len(current_path) - 1):
16            cost += graph[current_path[i]][current_path[i + 1]]
17
18        if cost < min_cost:
19            min_cost = cost
20            best_path = current_path
21
22    return best_path, min_cost
23
24
25 graph = [
26     [0, 10, 15, 20],
27     [10, 0, 35, 25],
28     [15, 35, 0, 30],
29     [20, 25, 30, 0]
```

Output

```
Best path: (0, 1, 3, 2, 0)
Minimum cost: 80

=== Code Execution Successful ===
```

main.py

Visualize

Run

```
1 from itertools import permutations
2
3 def assignment_problem(cost):
4     n = len(cost)
5
6     min_cost = float('inf')
7     best_assignment = None
8
9     for assignment in permutations(range(n)):
10        total = 0
11
12        for i in range(n):
13            total += cost[i][assignment[i]]
14
15        if total < min_cost:
16            min_cost = total
17            best_assignment = assignment
18
19    return best_assignment, min_cost
20
21
22 cost = [
23     [9, 2, 7],
24     [6, 4, 3],
25     [5, 8, 1]
26 ]
27
28 assignment, minimum = assignment_problem(cost)
29
```

Output

```
Assignment: (1, 0, 2)
Minimum cost: 9

=== Code Execution Successful ===
```

main.py

VisualizeRun

```
1 def knapsack(weights, values, capacity):
2     n = len(weights)
3
4     dp = [[0] * (capacity + 1) for _ in range(n + 1)]
5
6     for i in range(1, n + 1):
7         for w in range(capacity + 1):
8
9             if weights[i - 1] <= w:
10                dp[i][w] = max(
11                    values[i - 1] + dp[i - 1][w - weights[i - 1]],
12                    dp[i - 1][w]
13                )
14            else:
15                dp[i][w] = dp[i - 1][w]
16
17     return dp[n][capacity]
18
19
20 weights = [2, 3, 4, 5]
21 values = [3, 4, 5, 6]
22 capacity = 5
23
24 print("Maximum value:", knapsack(weights, values, capacity))
```

Output

Maximum value: 7

=== Code Execution Successful ===

main.py

VisualizeRun

```
1 def bubble_sort(a):
2     n = len(a)
3
4     for i in range(n):
5         for j in range(0, n - i - 1):
6
7             if a[j] > a[j + 1]:
8                 a[j], a[j + 1] = a[j + 1], a[j]
9
10    return a
11
12
13 a = [64, 34, 25, 12, 22, 11, 90]
14
15 print("Sorted list:", bubble_sort(a))
```

Output

Sorted list: [11, 12, 22, 25, 34, 64, 90]

=== Code Execution Successful ===

main.py

VisualizeRun

```
1 def find_max(a):
2     maximum = a[0]
3
4     for i in range(1, len(a)):
5         if a[i] > maximum:
6             maximum = a[i]
7
8     return maximum
9
10
11 a = [10, 25, 5, 78, 32]
12
13 print("Maximum element:", find_max(a))
```

Output

Maximum element: 78

=== Code Execution Successful ===

main.py		Visualize	Run	Output
<pre>1 def sort_and_find_max(a): 2 a.sort() 3 4 print("Sorted list:", a) 5 print("Maximum element:", a[-1]) 6 7 8 a = [45, 12, 78, 34, 90, 23] 9 10 sort_and_find_max(a)</pre>				Sorted list: [12, 23, 34, 45, 78, 90] Maximum element: 90 === Code Execution Successful ===

main.py		Visualize	Run	Output
<pre>1 def binary_search(a, key): 2 low = 0 3 high = len(a) - 1 4 5 while low <= high: 6 mid = (low + high) // 2 7 8 if a[mid] == key: 9 return mid 10 11 elif a[mid] < key: 12 low = mid + 1 13 14 else: 15 high = mid - 1 16 17 return -1 18 19 20 a = [10, 20, 30, 40, 50, 60] 21 key = 40 22 23 result = binary_search(a, key) 24 25 if result != -1: 26 print("Element found at index:", result) 27 else: 28 print("Element not found")</pre>				Element found at index: 3 === Code Execution Successful ===

main.py

Visualize

Run

Output

```
1 def merge_sort(a):
2
3     if len(a) <= 1:
4         return a
5
6     mid = len(a) // 2
7
8     left = merge_sort(a[:mid])
9     right = merge_sort(a[mid:])
10
11     result = []
12     i = 0
13     j = 0
14
15     while i < len(left) and j < len(right):
16
17         if left[i] < right[j]:
18             result.append(left[i])
19             i += 1
20
21         else:
22             result.append(right[j])
23             j += 1
24
25     result.extend(left[i:])
26     result.extend(right[j:])
27
28     return result
```

Sorted list: [3, 9, 10, 27, 38, 43, 82]

=== Code Execution Successful ===

main.py

Visualize

Run

Output

```
1 def compare_strings(s1, s2):
2
3     stack1 = list(s1)
4     stack2 = list(s2)
5
6     if len(stack1) != len(stack2):
7         return False
8
9     while stack1:
10         if stack1.pop() != stack2.pop():
11             return False
12
13     return True
14
15
16 s1 = "hello"
17 s2 = "hello"
18
19 if compare_strings(s1, s2):
20     print("Strings are equal")
21 else:
```

Strings are equal

=== Code Execution Successful ===